



Федеральное государственное автономное образовательное

учреждение высшего образования

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Направление подготовки: 09.03.04 – Системное и прикладное программное
обеспечение

Дисциплина «Вычислительная математика»

Отчёт по лабораторной работе №2

Вариант - 6

Выполнил

Линейский Аким Евгеньевич

P3215

Проверил

Наумова Надежда Александровна

Санкт - Петербург 2026

Содержание

1. Содержание.....	2
2. Цель	3
3. Описание метода.....	Ошибка! Закладка не определена.
4. Исходный код программы.....	8
5. Результат работы.....	15
6. Вывод	15

Цель

Изучить численные методы решения нелинейных уравнений и их систем, найти корни заданного нелинейного уравнения/системы нелинейных уравнений, выполнить программную реализацию методов на языке Python.

Порядок выполнения работы

1. Вычислительная реализация задачи

1.1. Решение нелинейного уравнения

1.1.1. Отделить корни заданного нелинейного уравнения графически

1.1.2. Определить интервалы изоляции корней

1.1.3. Уточнить корни нелинейного уравнения с указанной точностью

1.2. Решение системы нелинейных уравнений

1.2.1. Отделить корни заданной СНЛУ графически

1.2.2. Решить СНЛУ с указанной точностью

2. Программная реализация задачи

2.1. Для нелинейных уравнений

2.1.1. Реализовать численные методы в виде отдельных подпрограмм/методов/классов

2.1.2. Реализовать ввод и валидацию данных пользователем

2.1.3. Организовать вывод графика функции на всём исследуемом интервале

2.2. Для систем нелинейных уравнений

2.2.1. Реализовать алгоритм решения СНЛУ

2.2.2. Реализовать пользовательский ввод

2.2.3. Организовать вывод графика функций

2.2.4. Организовать вывод данных вычислений

2.2.5. Проверить правильность решения системы нелинейных уравнений

Рабочие формулы используемых методов

Вид уравнения для вычислительной реализации:

$$2x^3 + 3.41x^2 - 23.74x + 2.95$$

Производная функции для вычислительной реализации: +

$$f'(x) = 6x^2 + 6.82x - 23.74$$

Вторая производная функции для вычислительной реализации:

$$f''(x) = 12x + 6.82$$

Расчетная формула метода Ньютона: $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$

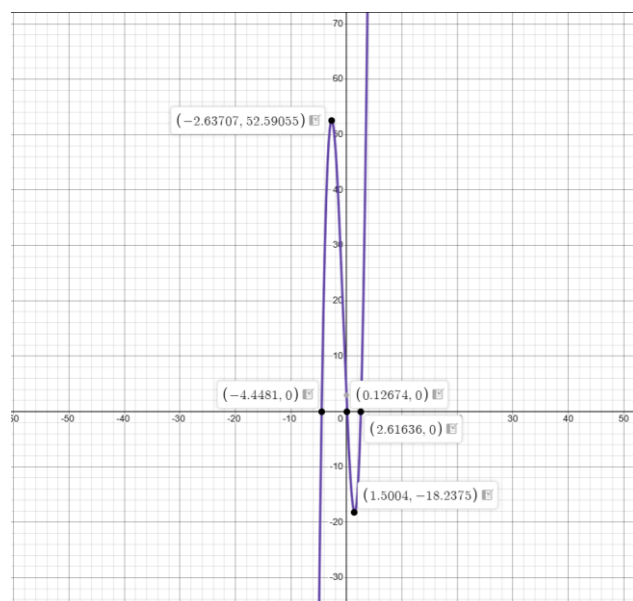
Расчетная формула метода хорд: $x_{k+1} = x_k - \frac{f(x_k) * (x_k - a)}{f'(x_k) - f(a)}$

Условие неподвижной точки для метода хорд: $f(c) * f''(c) > 0$

Расчетная формула метода простой итерации: $x_{k+1} = \frac{2x_k^3 + 3.41x_k^2 + 2.95}{23.74}$

Графики функций на исследуемом интервале

График функции $y = 2x^3 + 3.41x^2 - 23.74x + 2.95$



Таблицы вычислений

Вычисление корней производится с точностью $\varepsilon = 10^{-2}$

Для всех значений в таблице удерживать 3 знака после запятой

Вычисление крайнего правого корня методом Ньютона.

Интервал изоляции корня: $[2.5, 3]$

№	x_k	$f(x_k)$	$f'(x_k)$	x_{k+1}	$ x_{k+1} - x_k $
0	3	16.420	50.720	2.676	0.324
1	2.676	0.908	44.931	2.656	0.020
2	2.656	0.022	44.590	2.655	0.001

Вычисление крайнего левого корня методом хорд

Интервал изоляции корня: $[-5, -4]$

Выбор неподвижной точки:

На интервале $[-5, -4]$ вторая производная $f''(x) < 0$

$$f(-5) = 2 * (-125) + 3.41 * 25 - 23.74 * (-5) + 2.95 = -43.1$$

$$f(-4) = 2 * (-64) + 3.41 * 15 - 23.74 * (-4) + 2.95 = 24.47$$

По условию неподвижной точки для метода хорд: $b = -4$ подвижной

№	a	b	x	$f(a)$	$f(b)$	$f(x)$	$ x_{k+1} - x_k $
0	-5	-4	-4.362	-43.100	24.740	5.378	
1	-5	-4.362	-4.433	-43.100	5.378	1.154	0.071
2	-5	-4.433	-4.448	-43.100	1.154	0.237	0.015
4	-5	-4.448	-4.451	-43.100	0.237	0.050	0.003

Вычисление центрального корня методом простой итерации

Интервал изоляции корня: $[-0.5, 0.5]$

Проверка условия сходимости метода на выбранном интервале:

$$\varphi(x) = \frac{2x^3 + 3.41x^2 + 2.95}{23.74}$$

$$\varphi'(x) = \frac{6x^2 + 6.82x}{23.74}$$

Проверим значения производной на концах интервала:

$$\varphi'(0.5) = \frac{6 * 0.25 + 6.82 * 0.5}{23.74} \approx 0.207$$

$$\varphi'(-0.5) = \frac{6x^2 + 6.82x}{23.74} \approx -0.080$$

Т.к. на всём интервале $|\varphi'(x)| \leq 1$, то метод простой итерации будет сходиться.

Начальное приближение $x_0 = 0$

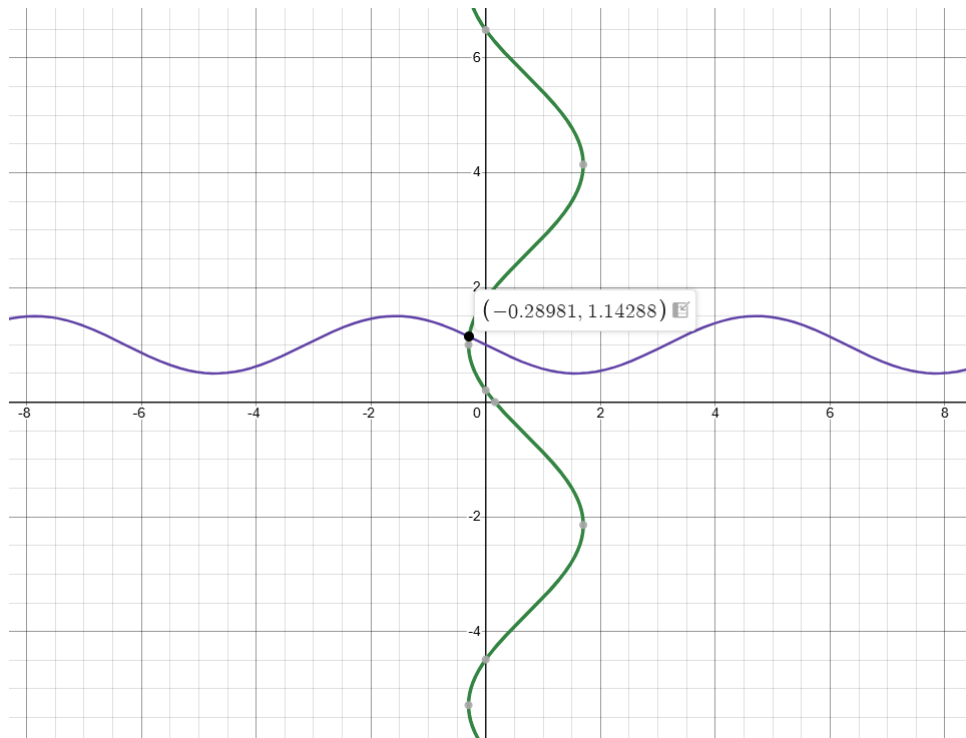
№	x_k	x_{k+1}	$f(x_{k+1})$	$ x_{k+1} - x_k $
1	0.000	0.124	0.006	0.124
2	0.124	0.127	-0.017	0.003

Решение СНЛУ

Решение системы нелинейных уравнений методом простой итерации

$$\begin{cases} \sin(x) + 2y = 2 \\ x + \cos(y - 1) = 0.7 \end{cases}$$

График функций системы



Решение

$$x \in [-0.5, 0], y \in [1, 1.5]$$

$$\begin{cases} x = \varphi_1(x, y) \\ y = \varphi_2(x, y) \end{cases}$$

$$x = 0.7 - \cos(y - 1) = \varphi_1(y)$$

$$y = 1 - 0.5\sin(x) = \varphi_2(x)$$

Проверка условия сходимости:

$$\left| \frac{\delta \varphi_1}{\delta x} \right| + \left| \frac{\delta \varphi_1}{\delta y} \right| = 0 + |\sin(y - 1)| < 1$$

На интервале $y \in [1, 1.5]$ максимум $|\sin(1.5 - 1)| \approx 0.479$

$$\left| \frac{\delta \varphi_2}{\delta x} \right| + \left| \frac{\delta \varphi_2}{\delta y} \right| = |-0.5\cos(x)| + 0 < 1$$

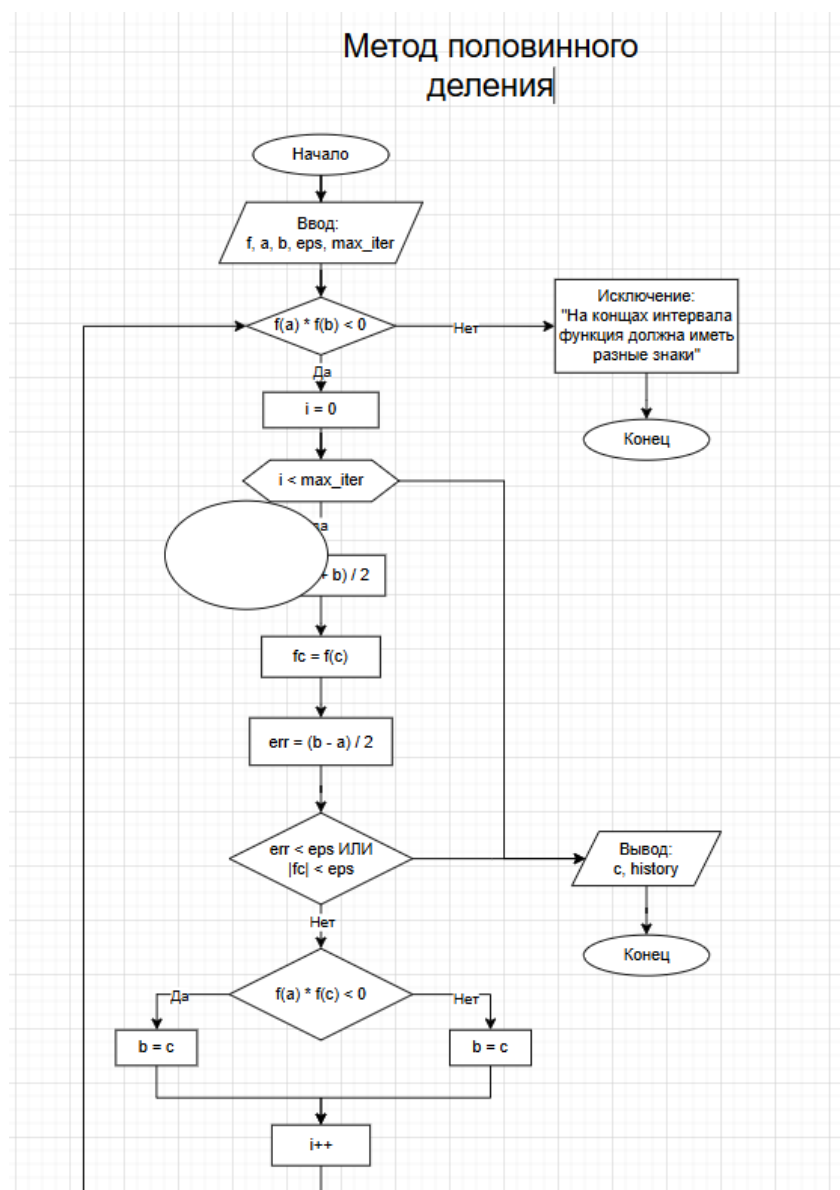
На интервале $x \in [-0.5, 0]$ максимум $|0.5\cos(0)| \approx 0.5$

Условие сходимости выполняется – метод сходится.

№	x_k	y_k	x_{k+1}	y_{k+1}	Δx
0	-0.250	1.250	-0.269	1.124	
1	-0.269	1.124	-0.292	1.133	0.023
2	-0.292	1.133	-0.291	1.144	0.011
3	-0.291	1.144	-0.290	1.144	0.001

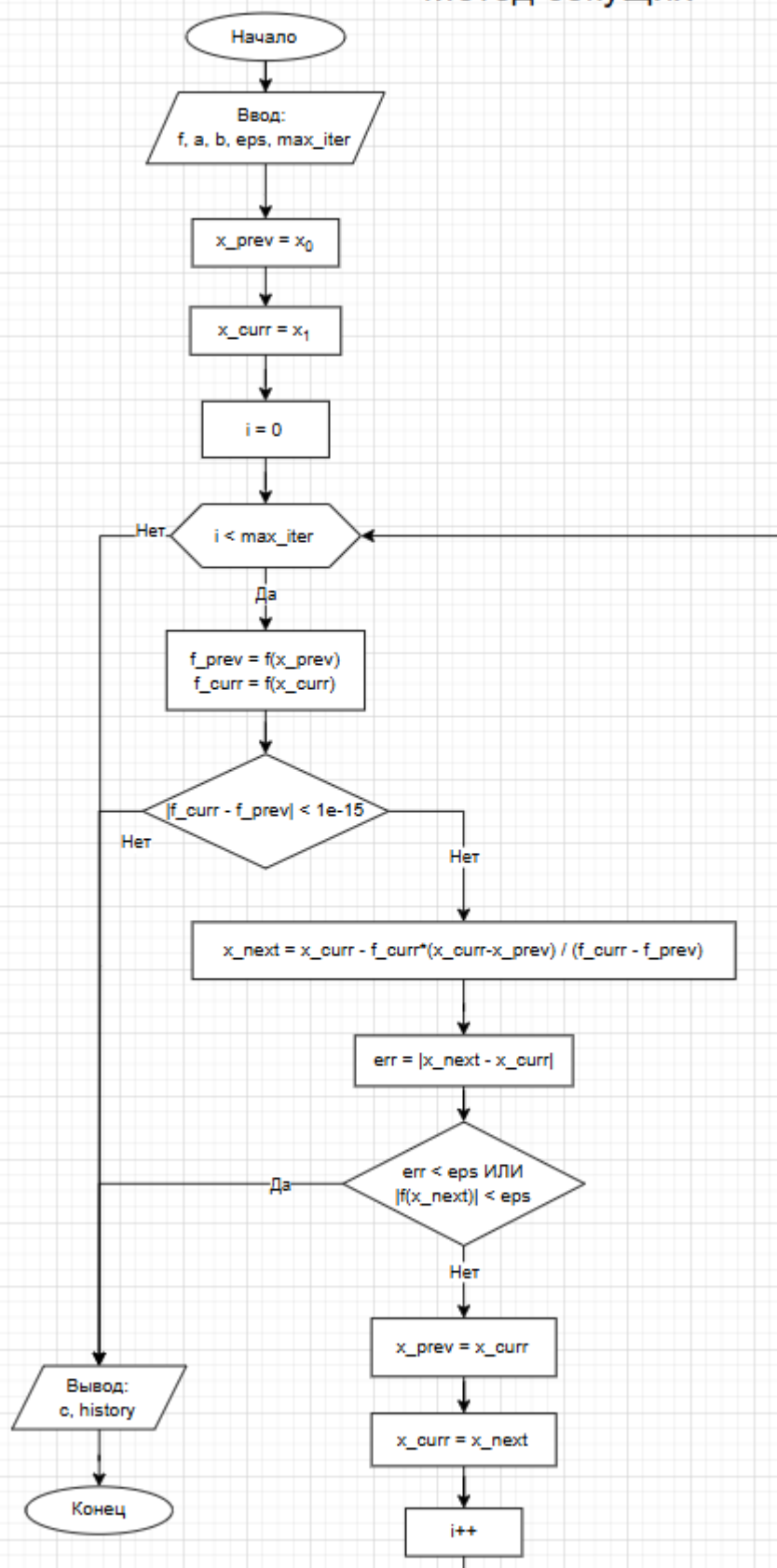
Блок схемы численных методов

Метод половинного деления



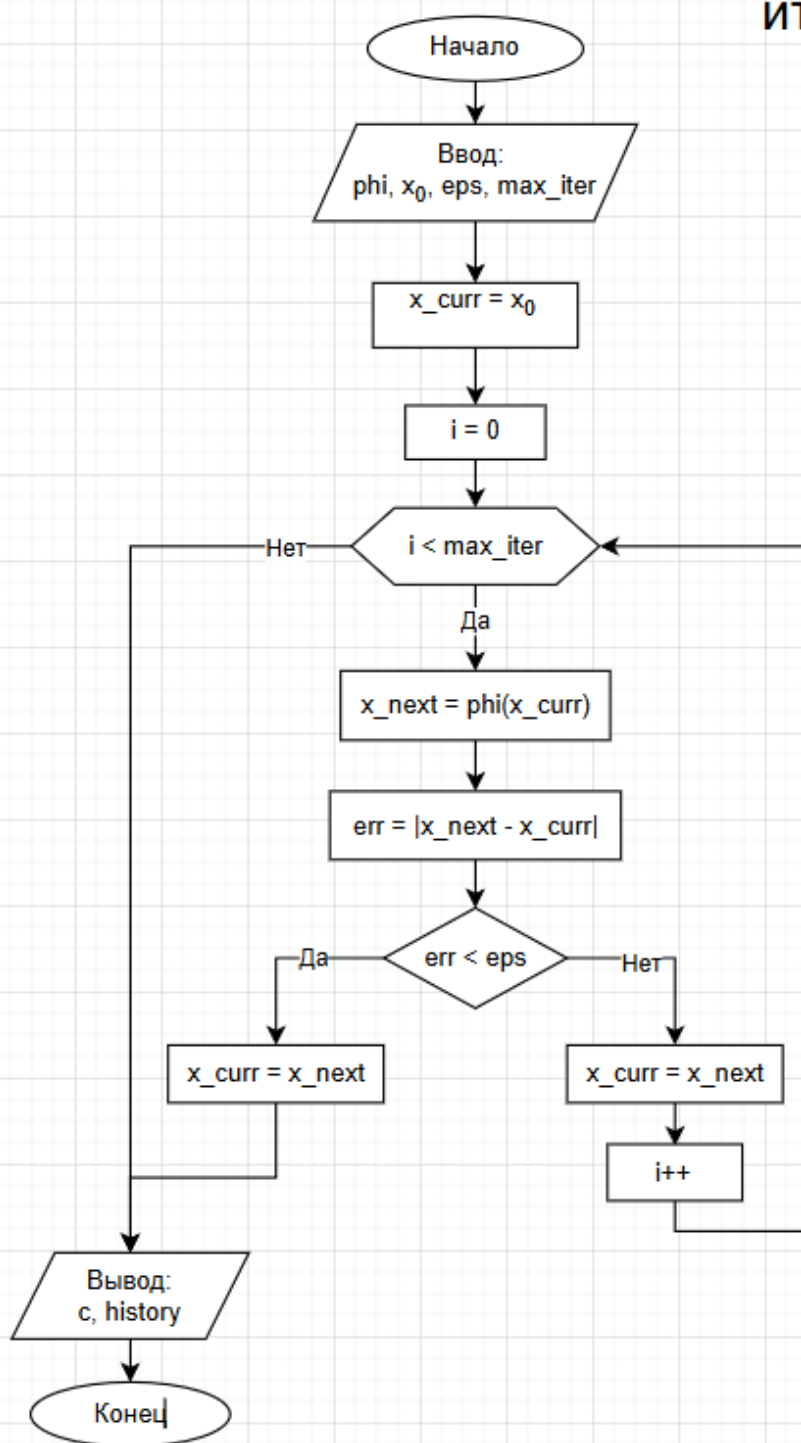
Метод секущих

Метод секущих



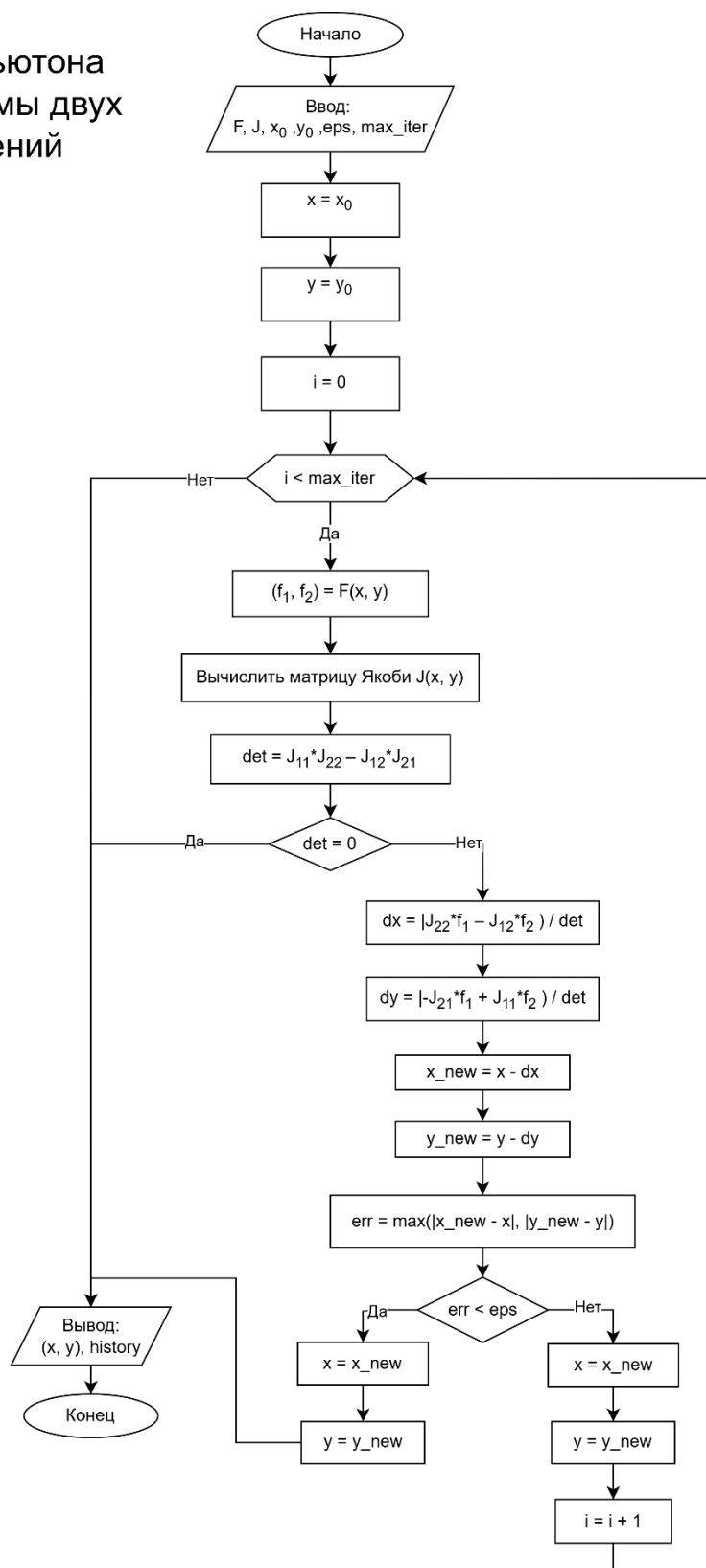
Метод простой итерации

Метод простой итерации



Метод Ньютона для систем двух уравнений

Метод Ньютона
для системы двух
уравнений



Исходный код вычислительных методов

```
# Методы решения уравнений
@staticmethod
def solve_bisection(f, a, b, eps, max_iter=100):
    history = []
    if f(a) * f(b) > 0:
        raise ValueError("На концах интервала функция должна иметь разные
знаки")

    for i in range(max_iter):
        c = (a + b) / 2
        fc = f(c)
        err = (b - a) / 2

        history.append({
            "iter": i+1,
            "a": a,
            "b": b,
            "x": c,
            "f(x)": fc,
            "error": err
        })

        if err < eps or abs(fc) < eps:
            break

        if f(a) * fc < 0:
            b = c
        else:
            a = c

    return c, history

@staticmethod
def solve_secant(f, x0, x1, eps, max_iter=100):
    history = []
    x_prev, x_curr = x0, x1

    for i in range(max_iter):
        f_prev = f(x_prev)
        f_curr = f(x_curr)

        if abs(f_curr - f_prev) < 1e-15:
            break

        x_next = x_curr - f_curr * (x_curr - x_prev) / (f_curr - f_prev)
        err = abs(x_next - x_curr)

        history.append({
```

```

        "iter": i+1,
        "x_prev": x_prev,
        "x_curr": x_curr,
        "f(x_prev)": f_prev,
        "f(x_curr)": f_curr,
        "x_next": x_next,
        "error": err
    })

    if err < eps or abs(f(x_next)) < eps:
        x_curr = x_next
        break

    x_prev, x_curr = x_curr, x_next

return x_curr, history

@staticmethod
def solve_simple_iteration(phi, x0, eps, max_iter=100):
    history = []
    x_curr = x0

    for i in range(max_iter):
        x_next = phi(x_curr)
        err = abs(x_next - x_curr)

        history.append({
            "iter": i+1,
            "x": x_curr,
            "phi(x)": x_next,
            "error": err
        })

        if err < eps:
            x_curr = x_next
            break

        x_curr = x_next

    return x_curr, history

# Метод решения систем
@staticmethod
def solve_newton_system(F, J, x0, y0, eps, max_iter=100):
    history = []
    x, y = x0, y0

    for i in range(max_iter):
        f1, f2 = F(x, y)
        jac = J(x, y)

        try:

```

```
        det = jac[0,0] * jac[1,1] - jac[0,1] * jac[1,0]
        dx = (jac[1,1] * f1 - jac[0,1] * f2) / det
        dy = (-jac[1,0] * f1 + jac[0,0] * f2) / det
    except:
        break

    x_new, y_new = x - dx, y - dy
    err = max(abs(x_new - x), abs(y_new - y))

    history.append({
        "iter": i+1,
        "x": x,
        "y": y,
        "f1(x,y)": f1,
        "f2(x,y)": f2,
        "error": err
    })

    if err < eps:
        x, y = x_new, y_new
        break

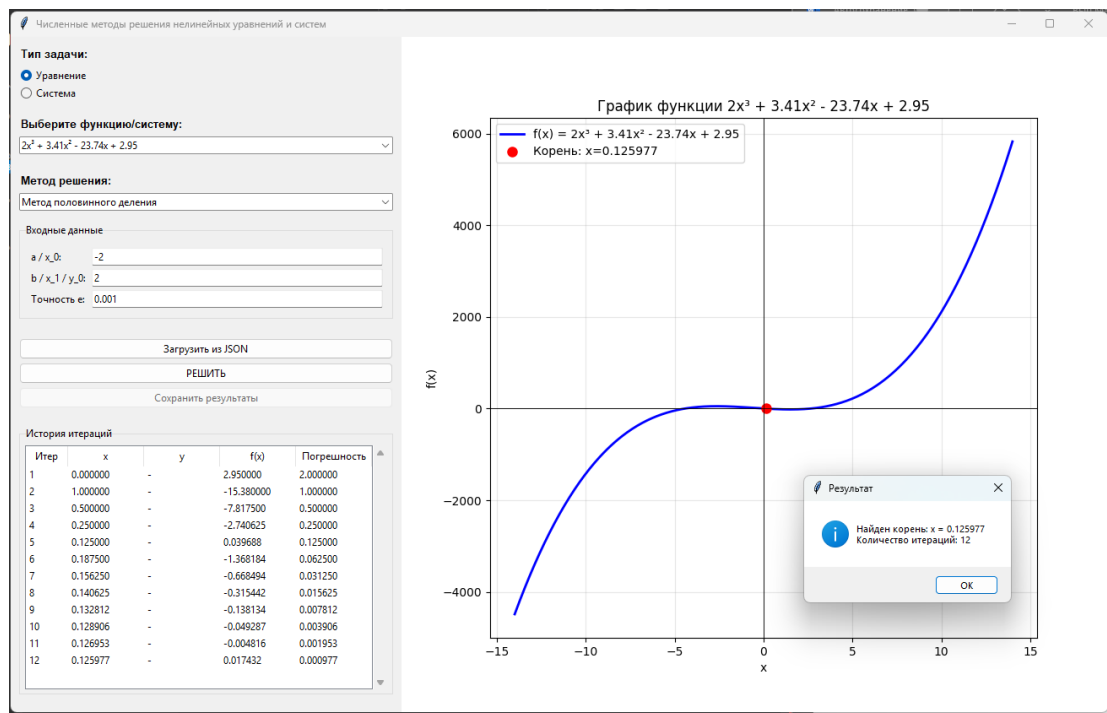
    x, y = x_new, y_new

return (x, y), history
```

Код доступен по ссылке на удаленный репозиторий GitHub:

[Код программы](#)

Результат работы



Вывод

Проделав данную лабораторную работу №2 я изучил способы решения нелинейных уравнений и систем уравнений, применил различные методы нахождения корней, реализовал численные методы и графический интерфейс с просмотром графика на языке Python.

